[BUG] With many observables `generate_shifted_tapes()` is called "unreas

https://github.com/PennyLaneAI/pennylane/issues/2430

ROW 25

[BUG] With many observables `generate_shifted_tapes()` is called "unreasonably often" resulting in massive performance loss #2430

New issue

Closed

cvjjm opened on Apr 8, 2022 · edited by cvjjm — Edits

Expected behavior

When taking the parameter shift hessian of a `QNode` returning the expectation value of several compatible observables, I expect the construction of shifted tapes to only take place once per gate, then the derivatives of all observables can be computed from this single set of shifted gates.

Actual behavior

In reality `parameter_shift.py:152:expval_param_shift` is called for each observable separately and `generate_shifted_tapes()` is therefore called number of parametrized gates times number of observables many times.

This results in a massive performance hit. In the 10 qubit example below, PennyLane spends over 60% of the time copying tapes, compared to just ~4% with the simulation of circuits, even on the slow `default.qubit` simulator.

Additional information

To get a nice graphical overview of what is happening run the example below with `cProfile`, e.g.. like this: `python -m cProfile -o timing test.py && gprof2dot -f pstats timing -o graph.dot && dot -Tsvg graph.dot -o graph.svg`

Source code

```
# test.py
# run with:
# python -m cProfile -o timing test.py && gprof2dot -f pstats timing -o graph.dot && dot -Tsvg graph.dot -o gr
import pennylane as qml
from pennylane import numpy as np

num_wires = 10
wires = range(num_wires)
dev = qml.device('default.qubit', num_wires)

np.random.seed(42)
init_params = np.random.randn(num_wires//2)

@qml.qnode(dev, diff_method='parameter-shift')
def qnode(params):
    for par_idx, wire in enumerate(wires[::2]):
        qml.Hadamard(wire)
        qml.CRY(params[par_idx], wires=[wire, wire+1])
    return [qml.expval(qml.PauliZ(wire1) @ qml.PauliZ(wire2)) for wire1 in wires for wire2 in wires if wire1 !

print(len([0 for wire1 in wires for wire2 in wires if wire1 != wire2]))

for _ in range(4):
    print(qml.jacobian(qnode)(init_params))
```

Tracebacks

No response

System information

```
Python 3.8.8 | packaged by conda-forge | (default, Feb 20 2021, 16:22:27)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import pennylane as qml; qml.about()
WARNING: pip is being invoked by an old script wrapper. This will fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead of running pip directly.
```

Assignees
No one assigned

Labels
bug

Type
No type

Projects
No projects

Milestone
No milestone

Relationships
None yet

Development
No branches or pull requests

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

Participants

```
License: Apache License 2.0
Location: /home/cvjjm/src/covqcstack/qcware/pennylane
Requires: numpy, scipy, networkx, retworkx, autograd, toml, appdirs, semantic-version, autoray, cachetools, pe
Required-by: pytket-pennylane, PennyLane-Qchem, PennyLane-Lightning, covvqetools
Platform info:           Linux-5.10.102.1-microsoft-standard-WSL2-x86_64-with-glibc2.10
Python version:          3.8.8
Numpy version:           1.20.1
Scipy version:           1.6.1
Installed devices:
- default.gaussian (PennyLane-0.21.0.dev0)
- default.mixed (PennyLane-0.21.0.dev0)
- default.qubit (PennyLane-0.21.0.dev0)
- default.qubit.autograd (PennyLane-0.21.0.dev0)
- default.qubit.jax (PennyLane-0.21.0.dev0)
- default.qubit.tf (PennyLane-0.21.0.dev0)
- default.qubit.torch (PennyLane-0.21.0.dev0)
- pytket.pytketdevice (pytket-pennylane-0.1.0)
- lightning.qubit (PennyLane-Lightning-0.20.2)
- cov.qubit (covvqetools-0.1.1)
```

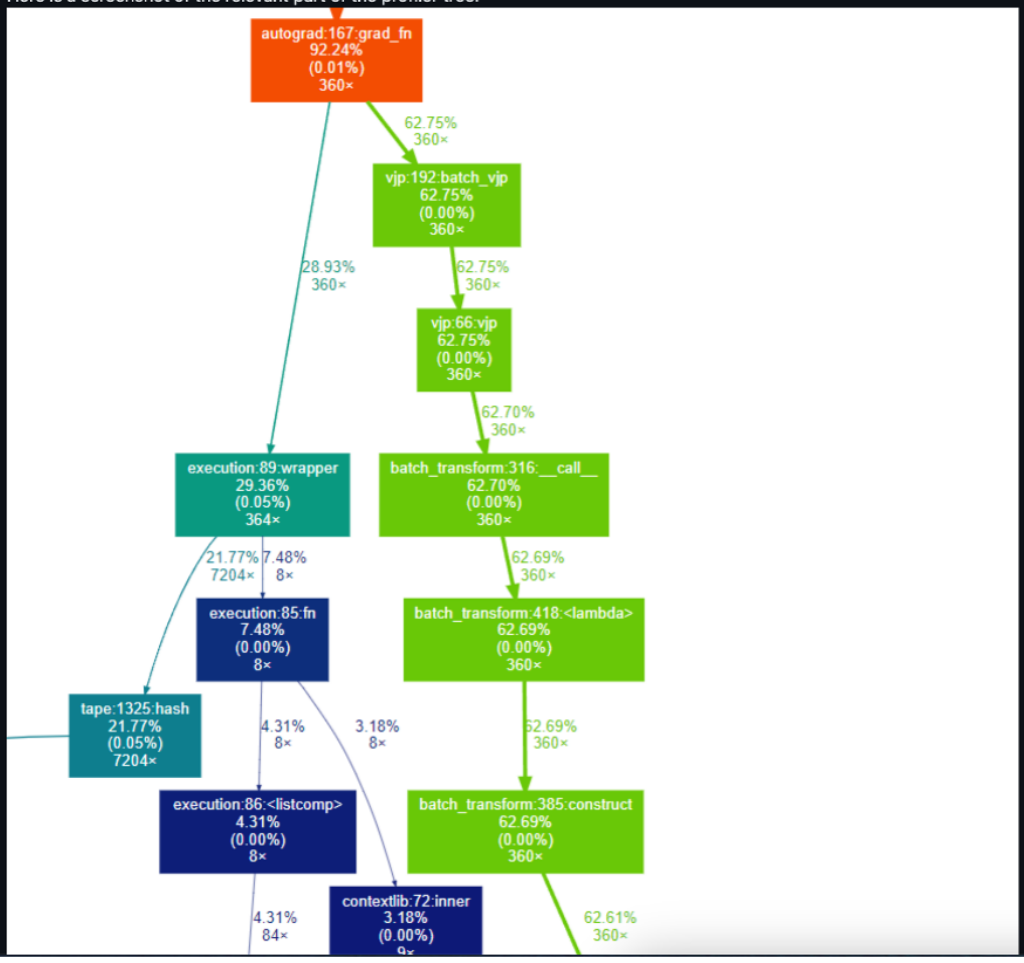
Existing GitHub issues

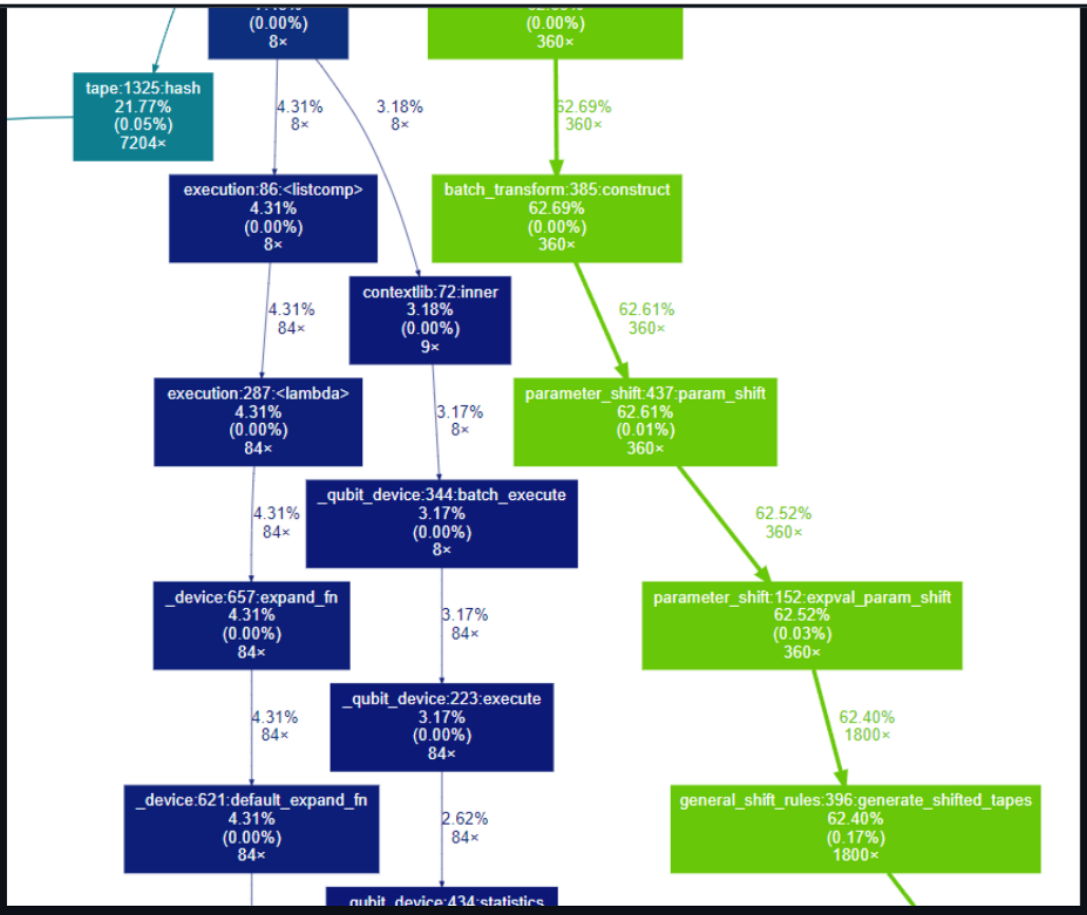
- [x] I have searched existing GitHub issues to make sure the issue does not already exist.

cvjjm added bug on Apr 8, 2022

cvjjm on Apr 8, 2022 — Contributor — Author

Here is a screenshot of the relevant part of the profiler tree:





cvjjm on Apr 8, 2022 — Contributor — Author

Also the amount of time spend on tape hashing seems excessive :-)

josh146 on Apr 8, 2022 — Member

Wow, thanks for alerting us on this **@cvjjm**!

> Also the amount of time spend on tape hashing seems excessive :-)

Yes, this one we are aware of and are thinking about how to fix! A solution probably involves smart usage of `lru_cache()`. We also noticed that calling of `np.round()` (as part of the caching) is *also* a significant overhead, so something we are looking into.

It is an interesting question, since the hashing results in a significant reduction in *quantum* resources (by reducing the amount of executions that take place), at a cost of additional classical resources. Something though that is very obvious in variational workflows!

cvjjm on Apr 8, 2022 — Contributor — Author

Reducing quantum resources should always have priority imho. We want to use PL for real experiments! :-)

cvjjm on Apr 8, 2022 — Contributor — Author

Reducing quantum resources should always have priority imho. We want to use PL for real experiments! :-)

mlxd on Apr 8, 2022 — Member

Hi @cvjjm this is really helpful. Running your provided example takes around 40s locally. Attacking the offending function allows me to bring this down to ~27s. It may take a few days, but I'll aim to put together a general solution that helps here.

🎉 2

mlxd mentioned this on Apr 11, 2022

NVIDIA cuQuantum support for QAOA using "lightning.gpu" #2429

[WIP] Add support for LRU caching of param-shifted tapes #2436

cvjjm mentioned this on Apr 20, 2022

Proposal for performance improvements in the Tape and Wires classes #2467

cvjjm on Apr 20, 2022 — Contributor — Author

Just a reminder from the discussion with @josh146 : We agree that while some smart caching can elevate the pain here and be very worthwhile overall, the core issue that should be addressed is that `generate_shifted_tapes()` should not be called num_params x num_observables many times but only num_params many times. Resolving this will dramatically improve performance for the test case above and generally help, whenever more than one observable is on a tape.

cvjjm mentioned this on Apr 20, 2022

Proposal for improvements in measurement and operation hashing #2468

josh146 on Apr 20, 2022 — Member

Thanks @cvjjm; as previously discussed it is potentially non-trivial as to *why* this is happening, since there is no explicit for loop over observables in the parameter-shift logic 🤔

My gut feeling is that it might be related to Hamiltonian differentiation. Currently, when PL identifies that the observable coefficients might be differentiable, this function is applied:

pennylane/pennylane/gradients/hamiltonian_grad.py
Line 19 in c272db0

```
19    def hamiltonian_grad(tape, idx):
```

which *does* loop over observables.

So the solution could be identifying why this is applied when it isn't needed, and fixing it.

cvjjm on May 19, 2022 — Contributor — Author

Here is some more statistics to highlight the severity of this:

- On PennyLane v0.19.0 executing the example above takes (most of this is startup time):

```
real    0m17.312s
user    0m8.488s
sys     0m4.016s
```

cvjjm on May 19, 2022 Contributor Author ···

Here is some more statistics to highlight the severity of this:

- On PennyLane v0.19.0 executing the example above takes (most of this is startup time):

```
real    0m17.312s
user    0m8.488s
sys     0m4.016s
```

- On PennyLane v0.23.0 it takes:

```
real    2m31.671s
user    1m37.670s
sys     0m51.943s
```

The other open question was, if I understand **@josh146** correctly, why `generate_shifted_tapes()` is called so often because there should be no loop over observables.

Putting into `generate_shifted_tapes()` a `traceback.print_stack()` gives some insight into what is happening:

```
  File "pl_test10.py", line 24, in <module>
    print(qml.jacobian(qnode)(init_params))
  File "/.../pennylane/pennylane/_grad.py", line 324, in _jacobian_function
    jac = tuple(_jacobian(func, arg)(*args, **kwargs) for arg in _argnum)
  File "/.../pennylane/pennylane/_grad.py", line 324, in <genexpr>
    jac = tuple(_jacobian(func, arg)(*args, **kwargs) for arg in _argnum)
  File "/.../autograd/autograd/wrap_util.py", line 20, in nary_f
    return unary_operator(unary_f, x, *nary_op_args, **nary_op_kwargs)
  File "/.../autograd/autograd/differential_operators.py", line 61, in jacobian
    return np.reshape(np.stack(grads), jacobian_shape)
  File "/.../autograd/autograd/numpy/numpy_wrapper.py", line 88, in stack
    arrays = [array(arr) for arr in arrays]
  File "/.../autograd/autograd/numpy/numpy_wrapper.py", line 88, in <listcomp>
    arrays = [array(arr) for arr in arrays]
  File "/.../autograd/autograd/core.py", line 14, in vjp
    def vjp(g): return backward_pass(g, end_node)
  File "/.../autograd/autograd/core.py", line 21, in backward_pass
    ingrads = node.vjp(outgrad[0])
  File "/.../autograd/autograd/core.py", line 67, in <lambda>
    return lambda g: (vjp(g),)
  File "/.../pennylane/pennylane/interfaces/autograd.py", line 188, in grad_fn
    vjp_tapes, processing_fn = qml.gradients.batch_vjp(
  File "/.../pennylane/pennylane/gradients/vjp.py", line 313, in batch_vjp
    g_tapes, fn = vjp(tape, dy, gradient_fn, gradient_kwargs)
  File "/.../pennylane/pennylane/gradients/vjp.py", line 182, in vjp
    gradient_tapes, fn = gradient_fn(tape, **gradient_kwargs)
  File "/.../pennylane/pennylane/transforms/batch_transform.py", line 330, in __call__
    return self._tape_wrapper(*targs, **tkwargs)(qnode)
  File "/.../pennylane/pennylane/transforms/batch_transform.py", line 418, in <lambda>
    return lambda tape: self.construct(tape, *targs, **tkwargs)
  File "/.../pennylane/pennylane/transforms/batch_transform.py", line 402, in construct
    tapes, processing_fn = self.transform_fn(tape, *args, **kwargs)
  File "/.../pennylane/pennylane/gradients/parameter_shift.py", line 667, in param_shift
    g_tapes, fn = expval_param_shift(tape, argnum, shifts, gradient_recipes, f0)
  File "/.../pennylane/pennylane/gradients/parameter_shift.py", line 240, in expval_param_shift
    g_tapes = generate_shifted_tapes(tape, idx, op_shifts, multipliers)
  File "/.../pennylane/pennylane/gradients/general_shift_rules.py", line 416, in generate_shifted_tapes
    traceback.print_stack(f=None, limit=None, file=None)
```

Can someone here who knows the inner guts of PL better than me make sense of this?

mlxd on May 19, 2022 · edited by mlxd Edits Member ···

Hi **@cvjjm** thanks for the above. I am currently going through this process, and there are a few things going on under the hood here with the 0.23.0 release. I plan to have a solution for some of these shortly.

josh146 on May 19, 2022 · edited by josh146 Edits Member

@cvjjm I'm digging into this in more detail, and I think this might be due to this issue in Autograd: https://stackoverflow.com/questions/54488875/improve-performance-of-autograd-jacobian

In particular, Autograd *itself* has a for-loop over the output shape of the cost function it is computing the Jacobian, which is quite inefficient. As far as I can tell, this doesn't happen in Torch, TensorFlow, or JAX.

For example, the following program is *very* fast, and doesn't have this for loop over observables:

```
import pennylane as qml
from pennylane import numpy as np
import jax
from jax import numpy as jnp

num_wires = 10
wires = range(num_wires)
dev = qml.device('default.qubit', num_wires)

np.random.seed(42)
init_params = np.random.randn(num_wires//2)

@qml.qnode(dev, diff_method='parameter-shift', interface="jax")
def qnode(params):
    for par_idx, wire in enumerate(wires[::2]):
        qml.Hadamard(wire)
        qml.CRY(params[par_idx], wires=[wire, wire+1])
    return [qml.expval(qml.PauliZ(wire1) @ qml.PauliZ(wire2)) for wire1 in wires for wire2 in wires if wire1 !

print(len([0 for wire1 in wires for wire2 in wires if wire1 != wire2]))


for _ in range(4):
    print(jax.jacobian(qnode)(init_params))
```

Then the question becomes: how come PennyLane v0.19 wasn't affected by this poor implementation of `autograd.jacobian`?

I think I can answer this: it is because, in PennyLane v0.19, we were attempting to hotfix this by caching the Jacobian *between* autograd calls! However, when we re-wrote PennyLane for arbitrary orders of differentiation, this hotfix became a memory leak and was removed.

josh146 on May 19, 2022 Member

It looks like attempts were made to improve the performance of `autograd.jacobian`, but as it is now no longer maintained, this will likely never happen: HIPS/autograd#524

josh146 on May 19, 2022 · edited by josh146 Edits Member

@mlxd @cvjjm here is the former hotfix, from PR #1131:

pennylane/pennylane/interfaces/autograd.py
Lines 198 to 204 in 8745c6f

```
198     # The following dictionary caches the Jacobian and Hessian matrices,
199     # so that they can be re-used for different vjp/vhp computations
200     # within the same backpropagation call.
201     # This dictionary will exist in memory when autograd.grad is called,
202     # via closure. Once autograd.grad has returned, this dictionary
203     # will no longer be in scope and the memory will be freed.
204     saved_grad_matrices = {}
```

It took advantage of closure in a really scary way.

It took advantage of closure in a really scary way.

In this particular case, my main suggestion would likely be to switch to JAX, or see if we could somehow modify `qml.jacobian` to compute the Jacobian via low-level Autograd functions in a way that is more efficient (see the code by **@twhughes** in the link above)

cvjjm on May 20, 2022 · edited by cvjjm — Edits — Contributor — Author

Looks like a perfectly valid use of closure to me :-)

A similar "caching" effect can also be used to speed up the above example (and still get the correct result). See here for a demo: `408ca1d`

10 pennylane/gradients/parameter_shift.py

```
@@ -144,6 +144,7 @@ def _get_operation_recipe(tape, t_idx, shifts, order=1):
144 144     return qml.math.stack([coeffs, mults, shifts]).T
145 145
146 146
    147 + result_cache = dict()
147 148 def expval_param_shift(tape, argnum=None, shifts=None, gradient_recipes=None, f0=None):
148 149     r"""Generate the parameter-shift tapes and postprocessing methods required
149 150     to compute the gradient of a gate parameter with respect to an
@@ -170,6 +171,13 @@ def expval_param_shift(tape, argnum=None, shifts=None, gradient_recipes=None, f0
170 171         list of generated tapes, in addition to a post-processing
171 172         function to be applied to the results of the evaluated tapes.
172 173     """
    174 +     global result_cache
    175 +     key = (tape, tuple(argnum), shifts, tuple(gradient_recipes), f0)
    176 +     print("aeaedaedaedx", key)
    177 +     if key in result_cache:
    178 +         print("Have seen this tape before!")
    179 +         return result_cache[key]
    180 +
173 181     argnum = argnum or tape.trainable_params
174 182
175 183     gradient_tapes = []
@@ -283,6 +291,8 @@ def processing_fn(results):
283 291
284 292         return qml.math.T(qml.math.stack(grads))
285 293
    294 +     result_cache[key] = (gradient_tapes, processing_fn)
    295 +
286 296     return gradient_tapes, processing_fn
```

This is obviously not a finished solution, one certainly needs to free the `result_dict` somehow at the right time, but it works and gives the correct results in the example above without the insane number of tape shift/copy/hash operations.

Maybe something along those lines can be added to PL?

And btw: Good job in reducing the hashing time. This already runs much faster with master than it did with 0.22.0, but it is still much slower than it should be...

josh146 on May 26, 2022 — Member

Thanks **@cvjjm!** I'll have a look into it. The subtlety of the previous hotfix (which I don't think is possible with the higher order diff support) is that the cache became automatically out of scope after Autograd backpropagation, which meant that Python was clearing the memory.

To replicate that, we might have to dig in and work out how to add the caching to https://github.com/PennyLaneAI/pennylane/blob/master/pennylane/interfaces/autograd.py.

Having said that, I think rewriting `qml.jacobian` to use a mix of jvp's and vjp's -- that is more efficient than simply looping over vjp's --- is the right way to go here, and should negate the need to cache altogether

josh146 on May 26, 2022 — Member

Thanks **@cvjjm**! I'll have a look into it. The subtlety of the previous hotfix (which I don't think is possible with the higher order diff support) is that the cache became automatically out of scope after Autograd backpropagation, which meant that Python was clearing the memory.

To replicate that, we might have to dig in and work out how to add the caching to https://github.com/PennyLaneAI/pennylane/blob/master/pennylane/interfaces/autograd.py.

Having said that, I think rewriting `qml.jacobian` to use a mix of jvp's and vjp's -- that is more efficient than simply looping over vjp's --- is the right way to go here, and should negate the need to cache altogether

cvjjm on Jul 4, 2022 — Contributor, Author

The solution you outlined with mixing of jvp's and vjp's sounds good! Is there any chance this could make it into one of the next releases?

antalszava on Jul 5, 2022 — Contributor

Hi **@cvjjm**, we have been looking into a fix to this issue in #2645. It would be using the caching approach. The PR needs a couple of final touches, but our plan is to have it in with the v0.25.0, in the next release. Would that work well?

cvjjm on Jul 5, 2022 — Contributor, Author

Great! The implementation in #2645 looks very nice.

🎉 2

mlxd mentioned this on Jul 12, 2022

Cache jacobian for reuse with param-shift #2645

antalszava on Jul 13, 2022 — Contributor

Hi **@cvjjm**, we've just merged in the fix. Let us know how it is in your use case. :)

cvjjm on Jul 13, 2022 · edited by cvjjm — Edits, Contributor, Author

Awesome! Just tested and current master now even beats the performance of v0.18.0 175.19s to 250.03s in a test that took several hours to run with v0.22 - v0.24 :-)

🎉 1

antalszava on Jul 13, 2022 — Contributor

That's awesome to hear! 🎉 :) Our plan now is to ship v0.25.0 around mid-August, this improvement would be attached to that release.

I'm closing this issue, but it can be re-opened if there is/will be relevance still.